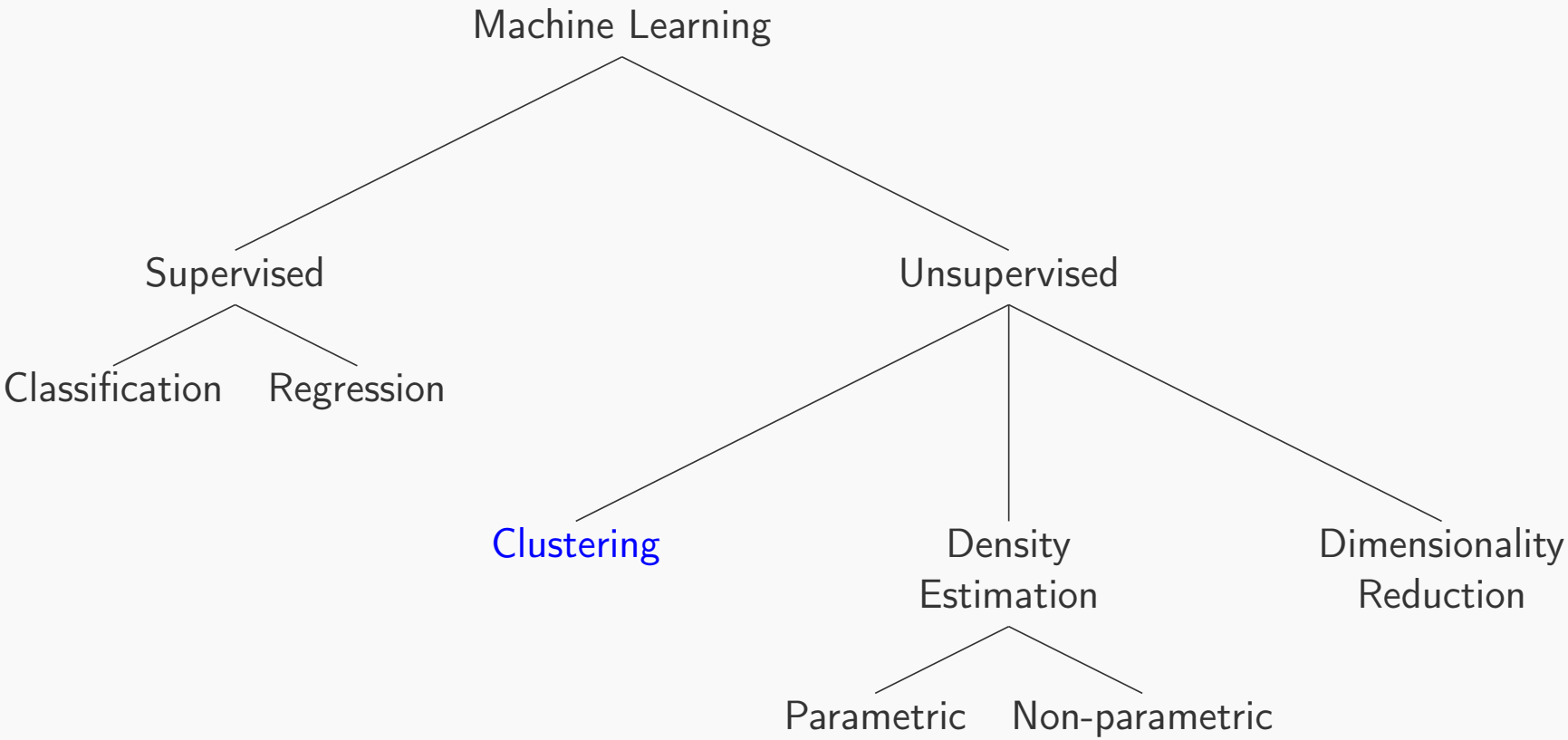
Machine Learning So Far



K-means Clustering

- Unsupervised learning algorithm to identify groups or clusters of similar data points in $\mathbb{R}^D$.
- Can be seen as an instance of the more powerful framework of Expectation Maximisation (to be covered later).
- Given data points $\{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$ and an integer $K > 1$, the *goal is to partition the data into K clusters*.
- Intuitively, clusters can be defined as having small intra-cluster and large inter-cluster distances.

K-means Clustering

- Define $\boldsymbol{\mu}_k$ as a representative vector of cluster k.
- Then we can compute the squared distance of any $\mathbf{x}_n$ from cluster k simply as

$$\|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2$$

- We also need a variable to denote assignment of $\mathbf{x}_n$ to the proper cluster.
- Define $\mathbf{r}_n$ using 1-of-K coding with $r_{nk} = 1$ if $\mathbf{x}_n$ belongs to cluster k and 0 otherwise.

K-means Clustering

- Then for a particular set of clusters $\{\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K\}$ and cluster assignments $\{\mathbf{r}_1, \ldots, \mathbf{r}_N\}$, we can compute the *sum-of-squared distances* between data points and their assigned clusters as

$$J(\{\boldsymbol{\mu}_k\}, \{r_{nk}\}) = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2$$

- Optimal set of clusters and assignments can be obtained via

$$\{\boldsymbol{\mu}_k\}^*, \{r_{nk}\}^* = \arg \min_{\{\boldsymbol{\mu}_k\}, \{r_{nk}\}} J(\{\boldsymbol{\mu}_k\}, \{r_{nk}\})$$

- Achieved via *iterative, alternating optimisation* between assignments $\{r_{nk}\}$ and clusters $\{\boldsymbol{\mu}_k\}$.

K-means Clustering

Data: Data points $\{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$, integer $K > 1$
Result: Cluster representatives $\{\boldsymbol{\mu}_k\}$, assignments $\{r_{nk}\}$
Choose some initial $\boldsymbol{\mu}_k$;
while *not converged* **do**
  Fix clusters and update assignments ($\{r_{nk}\} = \arg\min_{\{r_{nk}\}} J$);
  Fix assignments and update clusters ($\{\boldsymbol{\mu}_k\} = \arg\min_{\{\boldsymbol{\mu}_k\}} J$);
end

Algorithm 1: K-means Clustering

Take-home Quiz 6

- Show that the first minimisation amounts to updates

$$r_{nk} = \begin{cases} 1 & \text{if } k = \arg\min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2 \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

- Show that the second minimisation amounts to updates

$$\boldsymbol{\mu}_k = \frac{\sum_n r_{nk}\mathbf{x}_n}{\sum_n r_{nk}} \tag{2}$$

K-means Clustering

- The second minimisation just amounts to setting $\boldsymbol{\mu}_k$ to the mean of the data points assigned to cluster k. Hence the name *K-means*.
- Since objective function J is reduced at each iteration, convergence to a (local) minimum is guaranteed.
- For large/online datasets, there exists the *online K-means* algorithm with sequential updates

$$\boldsymbol{\mu}_k^{\text{new}} = \boldsymbol{\mu}_k^{\text{old}} + \eta_n(\mathbf{x}_n - \boldsymbol{\mu}_k^{\text{old}})$$

  with learning rate η_n that typically reduces with n.
- Replacing Euclidean distance $\|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2$ with a general disimilarity measure $\mathcal{V}(\cdot, \cdot)$ leads to the *K-medoids* algorithm.

K-means Clustering
Why alternating optimisation?

- Finding cluster centers and cluster memberships *simultaneously* is a *chicken-and-egg* problem.
- However, *individually* these problems are much simpler.
  - Given memberships, computing cluster centers is trivial.
  - Given cluster centers, computing memberships is trivial.
- **Alternating optimisation gives us a powerful framework of solving complex problems by decomposing them into simpler ones**.
- Notice that we appended the observed data $\mathbf{x}_n$ with some unobserved variables r_{nk} and then solved easy individual problems.
- These unobserved variables are called *hidden* or *latent* variables.

Alternating Optimisation for Latent Variable Models

Data: ...
Result: Optimal parameters
Choose some initial parameters;
while *not converged* **do**
    Fix parameters and update latent variables;
    Fix latent variables and update parameters;
end

Algorithm 2: Alternating Optimisation for Latent Variable Models

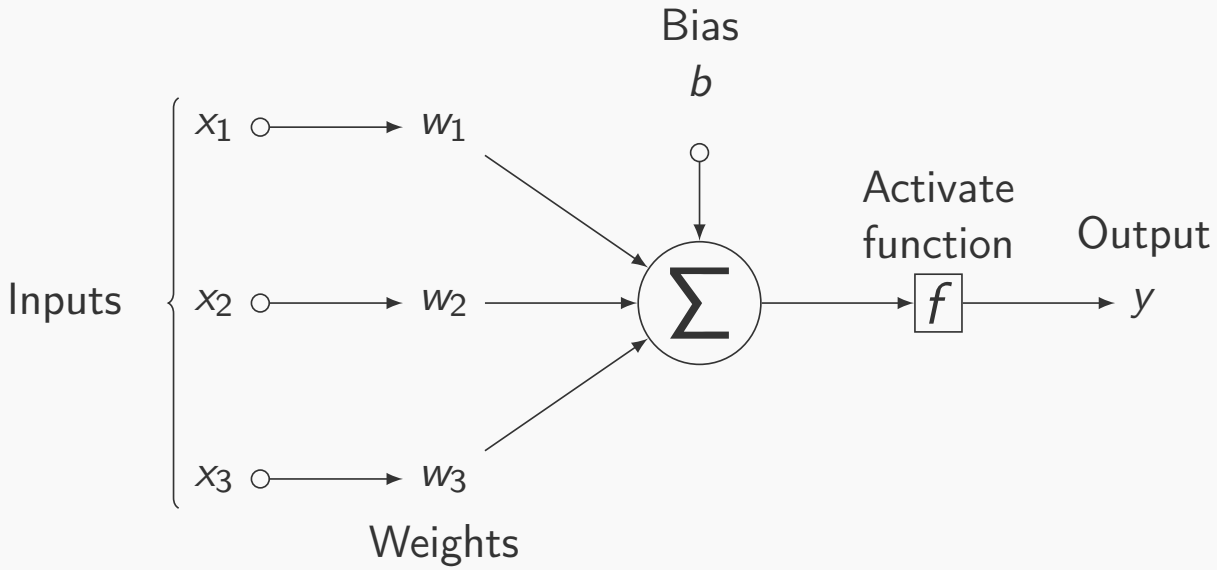
K-means Clustering
Disadvantages

- Assignment step has time complexity $O(NK)$.
  - Tree-based speed-ups exist.
  - Triangle inequality for distances can be exploited to avoid redundant distance computations.
- *Hard assignments* are not always the best option for points that lie near cluster boundaries.
  - Alternative is to use probability based *soft assignments*.
  - Leads to the framework of *mixture models*.

Neural Networks

- So far, we have learned $\mathbf{w}^*$ for mapping inputs $\mathbf{x} \in \mathbb{R}^D$ to targets $\mathbf{t}$.
- Often, working in a transformed space $\boldsymbol{\phi} \in \mathbb{R}^M$ makes it easier to learn the mapping.
- However, not all mappings are useful for the problem at hand. Is there an optimal mapping $\boldsymbol{\phi}^*$?
- Neural networks learn the optimal mapping $\boldsymbol{\phi}^*$ and also the optimal parameters $\mathbf{w}^*$.

Neural Networks
The Neuron



- The function of a biological neuron can be modelled as $y = f\left(\sum_j w_j x_j + b\right)$.

Neural Networks
The Neuron

- Model the output of the k-th neuron as $y_k = f(a_k) = f(\mathbf{w}_k^T \mathbf{x}) = f\left(\sum_j w_{kj} x_j\right)$ where
  - The x_j constitute values of input signals feeding into the neuron.
  - The w_{kj} are weights determining the importance given to input x_j by this neuron.
  - Dot-product $a_k = \sum_j w_{kj} x_j$ is called the *activation*.
  - $f(\cdot)$ is called the *activation function*. Determines behaviour of the neuron in response to its activation.
- The perceptron that we studied earlier is a very simple neuron model with f being the step function.

$$f(a) = \begin{cases} 1 & \text{if } a \geq 0 \\ 0 & \text{if } a < 0 \end{cases} \tag{1}$$